

Inversion of Control (инверсия управления) - происходит утеря контроля над выполнением кода (например, когда используют фреймворк).

Service Locator - вместо создания объектов («сервисов») напрямую с помощью "new", мы будем использовать специальный «фабричный» объект, который будет отвечать за создание, а точнее «нахождение» всех сервисов.

DI

Суть подхода: - Встроим (инъектируем) зависимость непосредственно в объект (компонент) - Используем интерфейс для обобщения принимаемых объектов

Dependency Injection Container хранит конфигурацию и умеет: - анализировать зависимости - создавать объекты с подстановкой зависимостей - вызывать методы с подстановкой зависимостей

GRASP (шаблоны распределения обязанностей)

1. Information Expert.

Обязанность назначается тому, кто владеет наибольшей информацией для её исполнения.

2. Creator.

Создавать экземпляры класса нужно там, где о нём знают.

3. Controller

Системные события должны быть собраны в отдельных классах-контроллерах, который делегирует обязанности другим классам.

4. Low Coupling (Низкая связанность)

Минимизируйте число связей между классами.

5. High Cohesion (Высокая целостность)

Обязанности класса должны быть тесно связаны между собой, иначе разбиты на отдельные классы.

6. Polymorphism

Для связи с каждой внешней системой должен быть свой интерфейс.

7. Pure Fabrication (Чистая выдумка)

Можно создавать классы, не отражающие реальные объекты, если требуется вынести лишние обязанности из другого класса.

8. Indirection (Перенаправление).

Создавайте посредника для исключения прямого воздействия одного элемента на другой.

9. Protected Variations (Защищенные изменения)

Отделяйте изменяемое от постоянного.

Service Locator

Service Locator - вместо создания объектов («сервисов») напрямую с помощью "new", мы будем использовать специальный «фабричный» объект, который будет отвечать за создание, а точнее «нахождение» всех сервисов.

Плюсы: - позволяет отвязать один компонент от другого

Минусы: - Локатор - глобальный объект - В Локатор можно записать всё что угодно и получить не те объекты

Реализация: - статич. сеттер для записи объектов внутрь локатора - стат. геттер

```
class Locator
{
    protected static $services = [];

    public static function setComponent($name, $object)
    {
        self::$services[$name] = $object;
    }

    public static function getComponent($name)
    {
        return self::$services[$name] ?? null;
    }
}
```

SOLID

SRP - single responsibility principle (принцип единственной ответственности)

Класс должен иметь одну причину для изменения. Каждый объект должен иметь одну обязанность.

Пример нарушения принципа: - размещать валидацию данных и загрузку файлов в одном классе

OCP - open–closed principle (п. открытости/закрытости)

Класс или модуль должен быть открыт для расширения, но закрыт для изменения.

LSP - Liskov substitution principle (п. подстановки Барбары Лисков)

Подклассы должны служить заменой для своих суперклассов.

Пример нарушения: в коде проверяется конкретный класс.

ISP - interface segregation principle (п. разделения интерфейсов)

Много узкоспециализированных интерфейсов лучше, чем один интерфейс общего назначения.

DIP - dependency inversion principle (п. инверсии зависимостей)

Должны быть зависимости на абстракциях, но не на что-то конкретное.